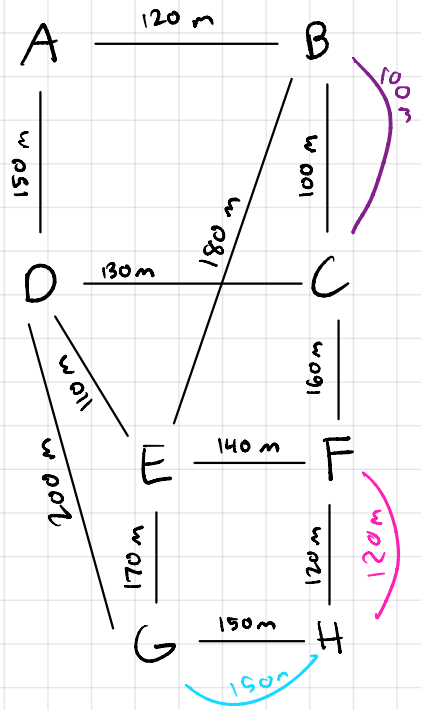
Graph Theory Assignment

SMRTTECH 4SC3
Nicolas Jao
400450151

Part A

a)



b)

	A	B	C	D	E	F	G	H
A		120		150				
B	120		100		180			
C		100		130		160		
D	150		130		110		200	
E		180		110		140	170	
F			160		140			120
G				200	170			150
H						120	150	

c) Degrees:

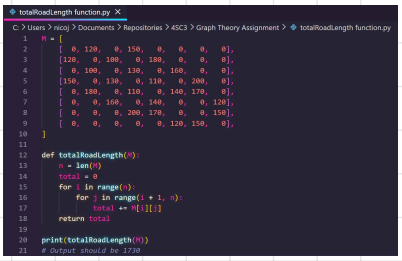
A = 2
B = 3
C = 3
D = 4
E = 4
F = 3
G = 3
H = 2

An Eulerian circuit needs all degrees to be even.

No Eulerian circuit exists.

Odd-degree vertices: B, C, F, G

$$M = \begin{bmatrix} 0 & 120 & 0 & 150 & 0 & 0 & 0 & 0 \\ 120 & 0 & 100 & 0 & 180 & 0 & 0 & 0 \\ 0 & 100 & 0 & 130 & 0 & 160 & 0 & 0 \\ 150 & 0 & 130 & 0 & 110 & 0 & 200 & 0 \\ 0 & 180 & 0 & 110 & 0 & 140 & 170 & 0 \\ 0 & 0 & 160 & 0 & 140 & 0 & 0 & 120 \\ 0 & 0 & 0 & 200 & 170 & 0 & 0 & 150 \\ 0 & 0 & 0 & 0 & 0 & 120 & 150 & 0 \end{bmatrix}$$



d) BC + FG = 100 + 120 + 150 = 370 (FG via E or H is equal)

BF + CG = 100 + 160 + 130 + 200 = 590 (BF via C is shortest, CG via D is shortest)

BG + CF = 180 + 170 + 160 = 510 (BG via E is shortest)

Minimum duplicated distance: 370 m

Duplicated roads: BC (100 m), FH (120 m), HG (150 m)

Total length: 120 + 150 + 100 + 180 + 130 + 110 + 160 + 140 + 200 + 170 + 150 + 120 + 370 = 2100 m

Optimal closed route: A - D - G - H - F - H - G - E - F - C - D - E - B - C - B - A

e) efficiency = $\frac{\text{required cleaning length}}{\text{total length travelled}} \times 100\% = \frac{1730}{2100} \times 100\% = 82.38\%$

Part B

f)

```python
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

# Since no strict output format has been instructed for this assignment,
# I have chosen to print the results to the console.

def streetCleanerRoute(M): # takes an input of a regular Python 2D list
    # Data preprocessing and visualization
    convert = np.array(M)
    adjacencyMatrix = nx.from_numpy_array(convert)
    pos = nx.spring_layout(adjacencyMatrix, seed=42)
    labels = {i: chr(65 + i) for i in adjacencyMatrix.nodes()}
    nx.draw(
        adjacencyMatrix,
        pos,
        with_labels=True,
        labels=labels,
        node_color="lightblue",
        node_size=800,
        font_size=12,
        font_weight="bold"
    )
    edgeLabels = nx.get_edge_attributes(adjacencyMatrix, "weight")
    nx.draw_networkx_edge_labels(
        adjacencyMatrix, pos, edge_labels=edgeLabels
    )
    plt.title("Original Street Network")
    plt.axis("off")
    plt.show()
    if nx.is_connected(adjacencyMatrix) == False:
        return "Error: Graph must be connected."

    # Odd-degree vertices
    oddDegreeVertices = []
    for node, degree in adjacencyMatrix.degree():
        if degree % 2 == 1:
            oddDegreeVertices.append(node)
    print("Odd degree vertices: ", oddDegreeVertices)

    # Shortest paths
    streets = nx.Graph()
    paths = []
    print("Shortest paths between odd vertices:")
    for x in range(len(oddDegreeVertices)):
        for y in range(x + 1, len(oddDegreeVertices)):
            u = oddDegreeVertices[x]
            v = oddDegreeVertices[y]
            path = nx.shortest_path(adjacencyMatrix, u, v, weight="weight")
            length = nx.shortest_path_length(adjacencyMatrix, u, v, weight="weight")
            streets.add_edge(u, v, weight=length)
            paths.append([u, v, path])
            print(" ", u, "to", v, ":", path, "cost", length)

    # Minimum-cost matching
    matching = nx.min_weight_matching(streets)
    print("Minimum-cost matching:")
    extraRoadLength = 0
    for pair in matching:
        a = pair[0]
        b = pair[1]
        cost = streets[a][b]["weight"]
        print(" ", a, "with", b, "cost", cost)
        extraRoadLength = extraRoadLength + cost
    print("Total extra distance from repeated edges:", extraRoadLength)

    # Making a copy of the graph with repeated edges
    newStreets = nx.MultiGraph(adjacencyMatrix)
    for pair in matching:
        a = pair[0]
        b = pair[1]
        path = []
        for entry in paths:
            if entry[0] == a and entry[1] == b:
                path = entry[2]
            if entry[0] == b and entry[1] == a:
                path = entry[2]
        for k in range(len(path) - 1):
            d = path[k]
            e = path[k + 1]
            f = adjacencyMatrix[d][e]["weight"]
            newStreets.add_edge(d, e, weight=f)

    # Eulerian route
    startingIntersection = 0
    roads = list(nx.eulerian_circuit(newStreets, source=startingIntersection))
    route = [startingIntersection]
    for road in roads:
        route.append(road[1])
    print("Eulerian route:", route)
    alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
    letter_route = []
    for intersection in route:
        letter = alphabet[intersection]
        letter_route.append(letter)
    print("Eulerian route (letters):", letter_route)

    # Cleaning efficiency
    totalRoadDistance = adjacencyMatrix.size(weight="weight")
    totalRoadDistanceTravelled = newStreets.size(weight="weight")
    efficiency = totalRoadDistance / totalRoadDistanceTravelled * 100
    print("Total distance of streets:", totalRoadDistance)
    print("Total distance travelled:", totalRoadDistanceTravelled)
    print("Efficiency:", efficiency)

# original graph:
M = [
    [_0, 120,  0, 150,  0,  0,  0,  0],
    [120,  0, 100,  0, 180,  0,  0,  0],
    [_0, 100,  0, 130,  0, 160,  0,  0],
    [150,  0, 130,  0, 110,  0, 200,  0],
    [_0, 180,  0, 110,  0, 140, 170,  0],
    [_0,  0, 160,  0, 140,  0,  0, 120],
    [_0,  0,  0, 200, 170,  0,  0, 150],
    [_0,  0,  0,  0,  0, 120, 150,  0],
]

print("ORIGINAL GRAPH:")
streetCleanerRoute(M)

# E-F closed:
M = [
    [_0, 120,  0, 150,  0,  0,  0,  0],
    [120,  0, 100,  0, 180,  0,  0,  0],
    [_0, 100,  0, 130,  0, 160,  0,  0],
    [150,  0, 130,  0, 110,  0, 200,  0],
    [_0, 180,  0, 110,  0,  0, 170,  0],
    [_0,  0, 160,  0,  0,  0,  0, 120],
    [_0,  0,  0, 200, 170,  0,  0, 150],
    [_0,  0,  0,  0,  0, 120, 150,  0],
]

print()
print("ROAD E-F CLOSED:")
streetCleanerRoute(M)
```

Results:

```
Run  streetCleanerRoute function
ORIGINAL GRAPH:
Odd degree vertices:  [1, 2, 5, 6]
Shortest paths between odd vertices:
  1 to 2 : [1, 2] cost 100
  1 to 5 : [1, 2, 5] cost 260
  1 to 6 : [1, 4, 6] cost 350
  2 to 5 : [2, 5] cost 160
  2 to 6 : [2, 3, 6] cost 330
  5 to 6 : [5, 7, 6] cost 270
Minimum-cost matching:
  5 with 6 cost 270
  2 with 1 cost 100
Total extra distance from repeated edges: 370
Eulerian route: [0, 3, 6, 7, 5, 7, 6, 4, 5, 2, 3, 4, 1, 2, 1, 0]
Eulerian route (letters): ['A', 'D', 'G', 'H', 'F', 'H', 'G', 'E', 'F', 'C', 'D', 'E', 'B', 'C', 'B', 'A']
Total distance of streets: 1730.0
Total distance travelled: 2100.0
Efficiency: 82.38095238095238

ROAD E-F CLOSED:
Odd degree vertices:  [1, 2, 4, 6]
Shortest paths between odd vertices:
  1 to 2 : [1, 2] cost 100
  1 to 4 : [1, 4] cost 180
  1 to 6 : [1, 4, 6] cost 350
  2 to 4 : [2, 3, 4] cost 240
  2 to 6 : [2, 3, 6] cost 330
  4 to 6 : [4, 6] cost 170
Minimum-cost matching:
  4 with 6 cost 170
  2 with 1 cost 100
Total extra distance from repeated edges: 270
Eulerian route: [0, 3, 6, 4, 6, 7, 5, 2, 3, 4, 1, 2, 1, 0]
Eulerian route (letters): ['A', 'D', 'G', 'E', 'G', 'H', 'F', 'C', 'D', 'E', 'B', 'C', 'B', 'A']
Total distance of streets: 1590.0
Total distance travelled: 1860.0
Efficiency: 85.48387096774194
```

matches answer on previous page

g) energy consumption : 0.25 kWh/km
usable battery capacity : 1.0 kWh
total distance required for full path in one go:
2100 m = 2.1 km
total energy required :
$(0.25)(2.1) = 0.525$
$0.525 < 1.0$
A single trip to clean all roads before needing to recharge is possible.

h)

```
Run  streetCleanerRoute function
ROAD E-F CLOSED:
Odd degree vertices:  [1, 2, 4, 6]
Shortest paths between odd vertices:
  1 to 2 : [1, 2] cost 100
  1 to 4 : [1, 4] cost 180
  1 to 6 : [1, 4, 6] cost 350
  2 to 4 : [2, 3, 4] cost 240
  2 to 6 : [2, 3, 6] cost 330
  4 to 6 : [4, 6] cost 170
Minimum-cost matching:
  4 with 6 cost 170
  2 with 1 cost 100
Total extra distance from repeated edges: 270
Eulerian route: [0, 3, 6, 4, 6, 7, 5, 2, 3, 4, 1, 2, 1, 0]
Eulerian route (letters): ['A', 'D', 'G', 'E', 'G', 'H', 'F', 'C', 'D', 'E', 'B', 'C', 'B', 'A']
Total distance of streets: 1590.0
Total distance travelled: 1860.0
Efficiency: 85.48387096774194

Process finished with exit code 0
```

Bonus:

If some roads are one way, the graph becomes directed, meaning roads can only be travelled in their permitted directions. Instead of finding odd-degree vertices and pairing them using minimum-cost matching, the algorithm must compare each vertex's in-degree and out-degree and find the minimum-cost directed paths to balance them. The goal remains to minimize the total distance travelled while covering every road and returning to the starting point, but the algorithm must respect the direction of each road when finding repeated paths and constructing the Eulerian circuit.

Practical changes:

- adjacency matrix would not be symmetrical
- odd-degree vertices calculation must be replaced with 'in-degree and out-degree calculation
- shortest paths must respect road directions
- minimum cost matching must be replaced by a new method

... and so on

Comparison Table:

	Original	E-F closed
Required Distance	1730 m	1590 m
Odd Vertices	B, C, F, G	B, C, E, G
Duplicated Roads	BC, FH, HG	BC, EG
Duplicated Distance	370 m	270 m
Total Travel	2100 m	1860 m
Cleaning Efficiency	82.38%	85.48%
Energy Usage	0.525 kWh	0.465 kWh